Hochschule
Augsburg University of
Applied Sciences

Abschlussdokumentation

Fakultät für
Informatik

Informatik (M.Sc.)

Chatter – Chatplattform für Teams

Roman Kolesnikov

Roman Kolesnikov
Kolbergstr. 18
86167 Augsburg
+49 17645624204
roman.kolesnikov1@hs-augsburg.de

Matrikelnummer:
2173133

Erstprüfer	:	Dipl.-Inf. (FH), Dipl.-Designer (FH), M.A. Erich Seifert
Eingereicht am	:	8. Januar 2024

Technische Hochschule Augsburg

An der Hochschule 1
D-86161 Augsburg

Telefon: +49 821 55860
Fax: +49 821 55863222
www.hs-augsburg.de
info@hs-augsburg.de

Inhaltsverzeichnis

1 Einführung **3**

2 Konzeption **4**

2.1 Projektplanung 4

2.2 Funktionalität 5

2.2.1 Nutzerverwaltung 5

2.2.2 Teams 6

2.2.3 Textkanäle 8

2.3 Technologien 9

2.3.1 Datenmodell 9

2.3.2 Architektur 10

2.3.3 Verwendete Technologien 11

3 Implementation **13**

3.1 Backend 13

3.1.1 Datenschicht 13

3.1.2 Businessschicht 14

3.1.3 Präsentationsschicht 16

3.1.4 Dependecy Injection 16

3.2 Frontend 17

3.3 Inbetriebnahme 18

4 Fazit **19**

Literatur **20**

1 Einführung

Asynchrone und digitale Kommunikationstechniken sind von der heutigen Welt nicht mehr wegzudenken, welche für Einsatzgebiete wie der Kommunikation zwischen Freunden bis hin zum Nutzen in großen Konzernen ragen. Durch die Internationalität in solchen großen Unternehmen werden Kommunikations-Möglichkeiten benötigt, welche den Austausch und das Festhalten von Informationen über Zeit und Ländergrenzen hinweg ermöglichen.

Auf dem Markt für solche Kommunikationsmittel stehen viele verschiedene Produkte zur Verfügung. Eines hierfür, welches eine große Beliebtheit erfährt und weitere Produkte maßgeblich beeinflusst hat, ist *Slack*. [12] Dieses ist im Kern ein Chatclient, welche mit verschiedenen Textkanälen in einem Team für die Kollaboration von Teammitgliedern arbeitet. So kann eine Organisation erstellt werden, welche alle Nutzer eines Unternehmens beingehören. In dieser Organisation stehen viele einzelne Textkanäle bereit, in welchen sie die Organisationsmitglieder austauschen können. Neben der Kommunikation in großen Kanälen gibt es auch die Möglichkeit als Person mit einzelnen Personen des Teams sich über private Nachrichten auszutauschen. Durch die Beliebtheit von *Slack* in Unternehmen sind auch weitere Chatclients, welche ein ähnliches Prinzip verfolgen, dies jedoch für andere Personengruppen anbieten, entstanden. Ein weiteres beliebtes Produkt wäre *Discord* [5], welches eher eine hohe private Nutzung erfährt.

So wurde in diesem Projekt ein ähnlicher Chatclient, namens „Chatter“, wie die bisher genannten Produkte, erstellt. Dieser verfolgt das Ziel eine Selbstorganisation von Nutzern in verschiedene einzelne Teams zu erlauben und so einen effizienten Kommunikationsweg für z. B. das Arbeiten in Gruppen zu ermöglichen. Jeder Nutzer kann dafür selbst eigene Teams erstellen und weitere Nutzer zu diesen einladen. In den Teams können über mehrere Kanäle Textnachrichten ausgetauscht werden.

2 Konzeption

Bevor mit der Entwicklung selbst angefangen werden kann, sollte zuerst das Konzept für die Applikation erstellt werden. Dies beinhaltet verschiedene Aspekte, wie die Planung des Projektes selbst, die Konzeption der Funktionalität der Applikation und die Technologien und Tools für die Entwicklung. Die folgenden Abschnitte sollen diese drei Beschriebenen Punkte genauer beleuchten.

2.1 Projektplanung

Das Projekt wurde alleine ohne ein Team entwickelt. Obwohl bei einer Arbeit alleine viel mehr Freiheiten offen sind als bei einer mit einem Team, sollte generell nicht ohne Struktur gearbeitet werden. Eine Festlegung einer Arbeitsweise verhilft dabei ein großes Projekt in kleine bearbeitbare Arbeitspakete aufzuteilen und diese systematisch bearbeiten zu können.

Für die Bearbeitung dieses Projektes wurde hierfür eine einfache, agile Arbeitsweise gewählt. So sollte das Projekt iterativ entwickelt werden, um schnell im Projekt einen arbeitsfähigen Prototyp zu erhalten. Hierfür sollten Arbeitspakete erstellt werden, welche jeweils ein einzelnes, abgeschlossenes Feature erhalten. Ein solches Arbeitspaket kann z. B. die Nutzerverwaltung sein. Solche Arbeitspakete können auch Abhängigkeiten untereinander haben, da z. B. das Erstellen eines Teams nicht ohne die Nutzerverwaltung möglich ist, ein solches Team immer an einen Nutzer gebunden ist und so die die Implementierung nicht möglich wäre. So kann auch eine sinnvolle Bearbeitungsreihenfolge für die Features bestimmt werden. An diesen Features kann Schritt für Schritt gearbeitet werden, bis ein „Minimal Viable Product“ (MVP) entsteht, mit welchem einem Nutzer die Kernfunktionalitäten des Produktes gegeben werden.

Nach diesen kann das Produkt weiter mit optionalen Features erweitert werden, die zwar für die Nutzer einen Mehrwert bieten, jedoch nicht essenziell für die Funktionalität sind.

Für die Planung eines solchen Projektes stehen dabei viele diverse Tools zur Verfügung. Um den Mehraufwand für die Projektplanung so gering wie möglich zu halten, wurde die integrierte Lösung von Gitlab [7] verwendet. So können zu den bisher beschriebenen Arbeitspakete sog. „Issues“ angelegt werden. So wurde bevor mit der Entwicklung eines solchen Features immer das jeweilige Issue erstellt, an dem gearbeitet werden sollte.

2.2 Funktionalität

Bevor die Technologien und die Architektur für das Projekt gewählt werden können, sollte nun das Ziel des Projektes und die Funktionalität dessen beschrieben werden. Damit können anhand der beschriebenen Anforderungen, besser die benötigten Technologien gewählt werden. Die folgenden Unterabschnitte sollen so die benötigten Funktionalitäten beschreiben, welche für eine Teambasierte Chatapplikation benötigt werden. Da kein separates Design bzw. Mockups für die Funktionalität erstellt wurden, werden die beschriebenen Anforderungen durch die schon implementierte Software illustriert.

2.2.1 Nutzerverwaltung

Die Nutzerverwaltung stellt eine grundlegende Funktionalität der Applikation dar, ohne die Teams und deren Unterkomponenten nicht erstellt werden können. Das wichtigste Ziel dieser ist es, dass neue Nutzer zum einen sich auf der Plattform registrieren können, als auch bestehenden Nutzern einen Login zur Plattform zu ermöglichen. Hierfür sollen in beiden Fällen Forms für beide Aktionen dem Nutzer gezeigt werden, wenn dieser in seinem Browser noch nicht eingeloggt ist. Die Nutzer können sich, wie es in der Abbildung 2.1 gezeigt wird, mit einer Kombination aus einem Nutzernamen und einem Passwort einen Login ausführen bzw. sich registrieren.

Eine weitere Funktionalität der Nutzerverwaltung ist, dass der Nutzer selbst Anpassungen an ihrem Account machen können. Da nur sehr wenige Daten

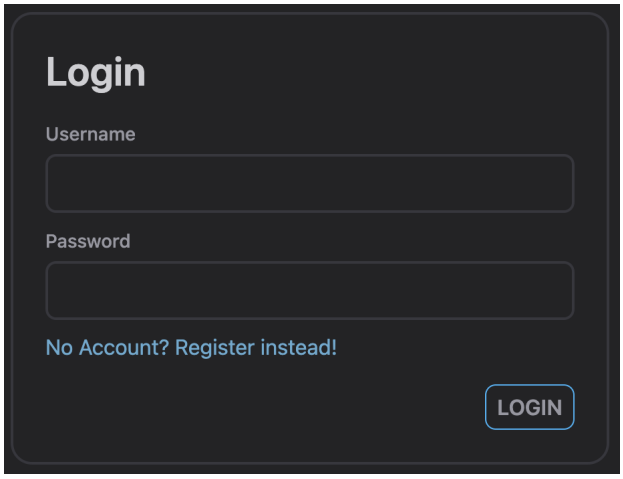

Abbildung 2.1: Login-Form

über den Nutzer gespeichert werden, beschränkt dies sich auf drei Aktionen. So kann ein Nutzer seinen Anzeigenamen für die Chats im Team bearbeiten können, welcher als Standard auf den Nutzernamen gesetzt ist. Auch sollte es die Möglichkeit geben sein Passwort ändern zu können als auch den Account endgültig zu löschen, wenn dieser nicht mehr benötigt wird. Diese Optionen sollten über die Einstellungen zu finden sein.

2.2.2 Teams

Nachdem ein Nutzer in der Plattform eingeloggt ist, sollte dieser nun die Möglichkeit haben, seine bisher erstellten Teams betrachten zu können und diese zu verwalten. Hierfür wird zuerst den Benutzer über in einer Seitenleiste alle seine Teams angeboten, welcher dieser selbst erstellt hat oder welche mit ihm geteilt wurden. Über ein Modal, dass durch die Seitenleiste erreicht werden kann, kann der Nutzer beide Aktionen durchführen. Zum Erstellen eines neuen Teams muss nur der Name für dieses angegeben werden. Wenn einen bestehenden beigetreten werden sollte, kann dies über einen Invite-Code erreicht werden.

Von der Seitenleiste aus kann zu einem spezifischen Team navigiert werden, wo man zum dem Hauptinterface der Applikation kommt. Diese ist eine drei geteilte Ansicht, welche die wichtigsten Informationen für ein Team zeigt. Die Abbildung 2.2 zeigt diese Ansicht. Diese zeigt die bisher erwähnte Seitenleiste zur Übersicht aller Teams. Neben dieser finden sich zwei weitere Seitenleisten,

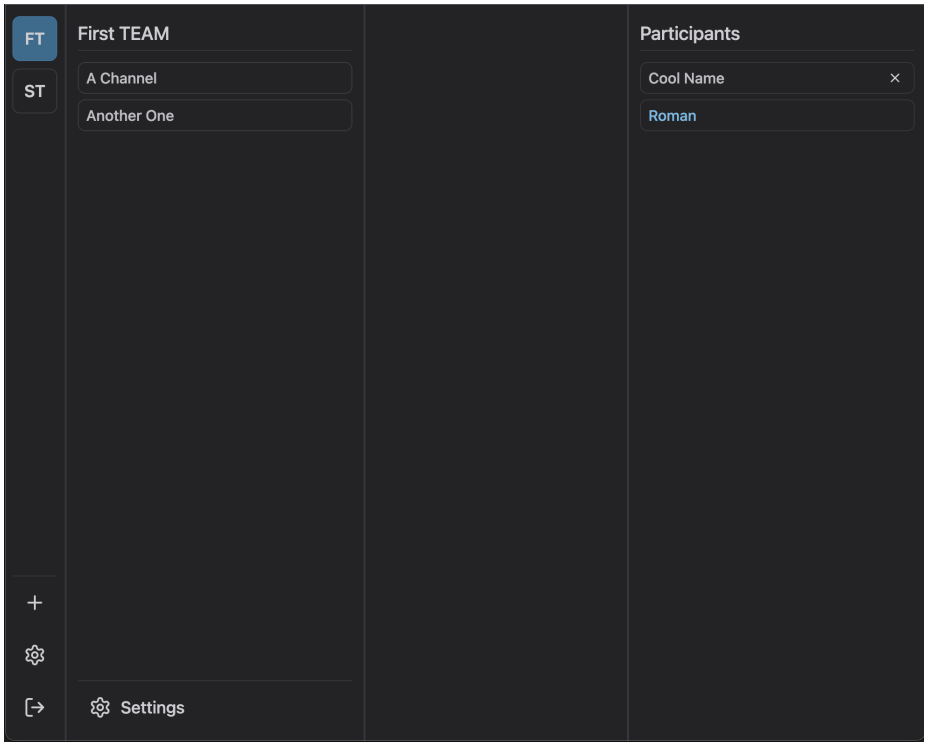

Abbildung 2.2: Teamübersicht

die zum Team gehören. In der linken Seitenleiste werden die Textkanäle angezeigt, welche wenn betreten, visuell markiert werden. Neben dieser erhält der Besitzer des Teams einen Button zu den Einstellungen eines Teams, während ein eigeladener Nutzer stattdessen einen Button zum Verlassen des Teams auffindet. In der weiteren Seitenleiste finden sich alle Teilnehmer des Teams. Hier wird der Besitzer des Teams speziell markiert. Auch erhält der Besitzer des Teams die Möglichkeit über diese Liste eingeladene Nutzer aus dem Team zu entfernen.

Auch sollte jedes Team seine eigenen Einstellungen haben. In diesen Einstellungen sollen zuerst allgemeine Optionen zu dem Team verwaltet werden. Dies beinhaltet zum bisherigen Zeitpunkt nur die Option den Namen des Teams zu ändern. Daneben können hier die Textkanäle verwaltet werden. Dafür kann ein neuer Kanal hinzugefügt und bestehende gelöscht werden. Die Option bestehende Kanäle nur umzubenennen, gibt es bisher nicht. Zuletzt sollen hier neue Invite-Codes für das Team generiert werden. Dieser kann der Besitzer des Teams erstellen und an weitere potentielle Nutzer geben, welche diese dann einlösen können. So ist eine einzelne Einladung nur einmal gültig.

2.2.3 Textkanäle

Die letzte Funktionalität stellen die Textkanäle, mit den Nachrichten in diesen dar. Wenn der Nutzer in einem Team zu einem Kanal navigiert, sollte dieser die Möglichkeit haben einzelne Textnachrichten zu senden und bereits gesendete zu betrachten. Dafür wird ein Chatfenster im bisher leeren Bereich zwischen den Seitenleisten, wie es in der Abbildung 2.3 zu sehen ist, im Team gezeigt, wenn auf einen der Kanäle in der Seitenleiste geklickt wird. Danach kann der Nutzer Nachrichten schreiben, die allen Nutzern zugestellt werden. Wenn der Nutzer eine Nachricht sendet und weitere Teilnehmer zurzeit denselben Kanal betrachten, wird diese Nachricht allen Teilnehmern in Echtzeit zugestellt und die Seite muss nicht neugeladen werden. Neben den Echtzeit-Nachrichten kann auch die Historie des Kanales mit den bisher gesendeten Nachrichten betrachtet werden. Diese sollten, damit der Nutzer nicht zu lange auf die Historie warten muss, in kleinen Paketen geladen werden. Wenn der Nutzer an das Ende der bisher geladenen Historie gescrollt hat, sollten diese falls noch Nachrichten vorhanden sind, nachgeladen werden.

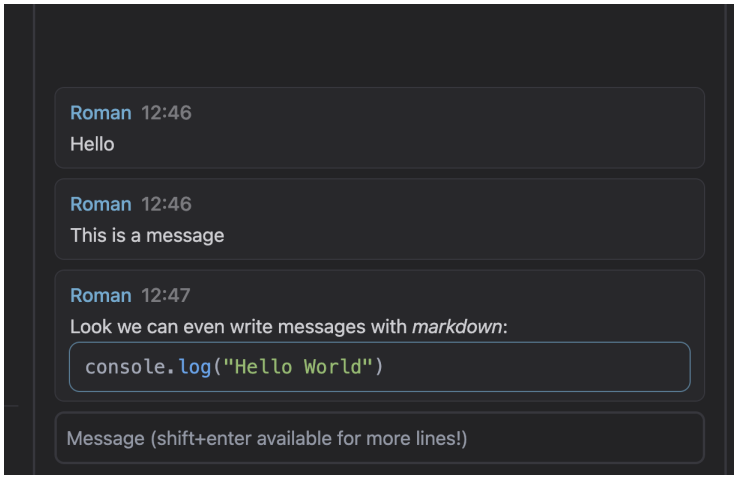

Abbildung 2.3: Textkanalnachrichten

Um die Nachrichten zu schreiben, wird ein Textfeld angeboten. In diesem sollten die Nachrichten einfach geschrieben werden und mit dem Drücken der Enter-Taste gesendet werden können. Da öfters auch längere Nachrichten gesendet werden müssten, kann mit der Kombination aus der Shift- und Enter-Taste eine neue Zeile begonnen werden. Das Textfeld wird sich hierfür auch automatisch in der Größe erweitern.

2.3 Technologien

Zuletzt sollen noch für die Konzeption die verwendeten Technologien, die Architektur und das Datenmodell der Applikation beschrieben werden. Diese sollen zeigen, nach welchen generellen Prinzipien das Projekt aufgebaut wurde.

2.3.1 Datenmodell

Da für dieses Projekt Nutzerdaten persistent gespeichert werden müssen, wird auch ein Datenmodell für dieses benötigt. Dabei wurde für die persistente Haltung der Daten sich für eine relationale Datenbank entschieden. Hierfür wurde die Datenbank PostgreSQL verwendet. Die Abbildung 2.4 zeigt dabei das Datenbankschema der Applikation. Da die Tabellen im Backend direkt auf Klassen gemapped werden und der Name beibehalten wird, haben auch die Tabellen hier eine „Entity“-Endung.

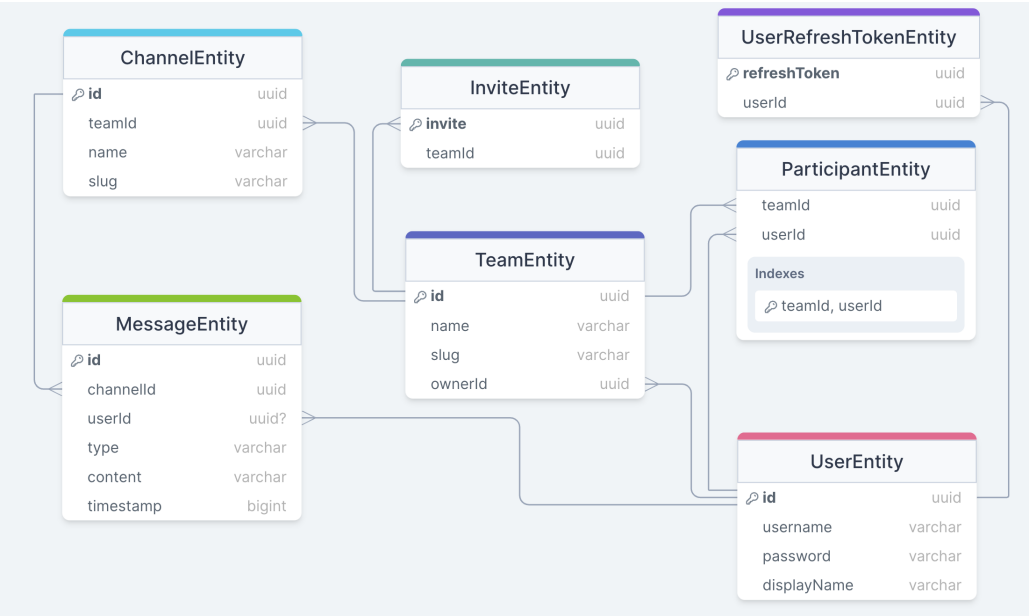

Abbildung 2.4: Datenbank-Schema

Für das Nutzerverwaltung werden zwei Tabellen benötigt, zum einen die *UserEntity* Tabelle als auch die *UserRefreshTokenEntity* Tabelle. Die *UserEntity* Tabelle hält den Nutzer selbst. Dies beinhaltet den einen Nutzernamen für den Login, ein Passwort, welches zuvor gehashed wird, und einen Anzeigenamen für die Teilnehmerliste und Chatnachrichten. Die *UserRefreshTokenEntity* Tabelle

wird zusätzlich benötigt, um die *refreshTokens* für die spätere Authentifikation zu speichern.

Die weiteren Tabellen werden zur Verwaltung der Teams und der Unterkomponenten gebraucht. Die *TeamEntity* Tabelle, speichert den Namen des Teams und einen *slug*, welcher später im Browser für eine lesbare Identifikation in der URL verwendet wird. Ein Team ist immer an einem Besitzer verknüpft. Um weitere Nutzer zu einem Team einzuladen, werden die Einladungscodes in der *InviteEntity* Tabelle gespeichert. Nach einem erfolgreichen Beitritt zu einem Team wird die Zugehörigkeit über die *ParticipantEntity* Tabelle festgehalten, in der Nutzer zu Teams zugeordnet werden. Weiterhin die verschiedenen Textkanäle in der *ChannelEntity* Tabelle gespeichert und besitzen auch wie ein Team selbst, *slugs* für die Verwendung im Browser. Zuletzt gibt es noch eine Tabelle für die Textnachrichten. In diesen wird der der Inhalt einer Nachricht, der Verfasser und ein Zeitstempel, wann die Nachricht geschrieben wurde, persistiert. Für den Inhalt der Nachricht gibt es zwei Spalten die *type* und *content* Spalte. Dies wurde dafür verwendet, um eine zukünftige Erweiterbarkeit vorzunehmen, wo z. B. auch neben einfachen Textnachrichten Bilder oder andere Nachrichtentypen gesendet werden können.

2.3.2 Architektur

Zur Entwicklung des Projektes wird noch die generelle Architektur für die gesamte Applikation benötigt. Da es sich hier um eine Webapplikation handelt wurde eine Client-Server-Architektur gewählt. Dies soll heißen, dass es einen Server (im Folgenden auch „Backend“ genannt) gibt, welcher die Daten und die Domänenlogik der Applikation hält. Dieser selbst bietet keine Interaktivität für den Nutzer. Die Interaktivität kommt durch die einzelnen Clients (im Folgenden auch „Frontend“ genannt), die die Nutzer bedienten können Zustande. Da eine Webapplikation entwickelt wird, sind Clients die Webbrowser der Benutzer. Ein Client bietet so die Interaktivität und kommuniziert zum Erhalt der Daten und zur Manipulation dieser mit dem Server.

Der Server folgt dabei einer einfachen Schichtenarchitektur. In dieser werden die Dienste einer Applikation in mehrere Schichten aufgebaut, welche aufeinander aufbauen. So wird die Funktionalität der vorherigen Schicht verwendet, um diese zu erweitern und eigene Funktionalität in einer hinzuzufügen.

(vgl. [1, S. 35]) Dabei wurde das Backend in drei Schichten gegliedert: die Datenschicht, die Businessschicht und die Präsentationsschicht. Durch die Datenschicht werden wird ein Zugang zu den Daten in der Datenbank und weiteren Persistenztechnologien ermöglicht. Diese können durch die Schicht erhalten und manipuliert werden. In der Businessschicht wird die eigentliche Applikationslogik erstellt. Dies beinhaltet z. B. die Erstellung eines Nutzers oder das Senden von Textnachrichten. Zuletzt bietet die Präsentationsschicht eine REST-Schnittstelle für die Interaktivität des Frontends und eine zusätzliche Websocket-Schnittstelle, um Echtzeit-Benachrichtigungen zu erhalten.

Das Frontend selbst wird durch eine Single Page Applikation (SPA) gebildet. In dieser wird das User-Interface (UI) dynamisch komplett im Browser generiert. Dafür wird eine minimale HTML-Datei ausgegeben, welche den gesamten Quelltext für das Bilden des eigentlichen User-Interfaces und der Interaktion mit dem Backend enthält. Der Client kann über asynchrone Request im Hintergrund Daten vom Backend erhalten und die UI dementsprechend anpassen. Bei Aktionen des Nutzers, wie z. B. dem Erstellen eines neuen Projektes oder seinem Login, können auch Anfragen an den Server gesendet werden, um diese auszuführen.

2.3.3 Verwendete Technologien

Zuletzt sollten in der Konzeption, die verwendeten Technologien für das Projekt beschrieben werden. Hierfür wird zwischen dem Frontend und Backend unterschieden, da dafür zwei verschiedene Programmiersprachen verwendet wurden. Die verwendete relationale Datenbank für die Datenhaltung wurde dabei schon in dementsprechenden Kapitel beschrieben.

Für die Entwicklung des Backends wurde die Programmiersprache Kotlin einer Sprache, die auf der JVM verwendet werden kann, und deren Ökosystem verwendet. Die wichtigsten Pakete waren hierfür: *sqldelight* [2], *ktor* [8] und *dagger* [4] zusammen mit *anvil* [14]. *sqldelight* bietet dafür einen Typsicheren Zugriff zur Datenbank, welche über Quelltext-Generation geschieht. Durch *ktor* konnte die REST-Schnitstelle als auch das Behandeln von Websocket-Verbindungen für die Präsentationsschicht implementiert werden. Zuletzt konnte über die Bibliothek *dagger* und dem Erweiterungspaket *anvil* Paket, eine

automatische Dependency-Injection für die Verbindung der einzelnen Schichten und deren Diensten ermöglicht werden.

Das Frontend wurde mithilfe von TypeScript und der UI-Bibliothek *SolidJS* [13] entwickelt. Dabei handelt es sich um eine komponentenbasierte UI-Bibliothek, durch die einzelne UI-Elemente mithilfe von einfachen Funktionen beschrieben werden können. Neben dieser wurde zwei kleinere Bibliothek, welche die Funktionalität von *SolidJS* erweitern, verwendet.

3 Implementation

Mit der beschriebenen Konzeption kann nun die Implementation des Webapplikation besprochen werden. Diese wird dabei in das Frontend und Backend aufgeteilt. So wurde zwar immer Feature basiert gearbeitet, d. h. um eine Funktionalität zu erweitern mussten meistens sowohl das Frontend als auch das Backend angepasst werden, jedoch hat man eine starke Konzeptionelle Unterscheidung, wie beide Applikationsteile aufgebaut sind. So sollen in den folgenden Abschnitten zuerst das Backend und danach das Frontend beschrieben werden. Zuletzt soll noch eine kurze Übersicht gegeben werden, wie die Webapplikation ausgeführt werden kann.

3.1 Backend

Das Backend wurde, wie schon in der Konzeption erläutert, mithilfe von der Programmiersprache *Kotlin* geschrieben und folgt einer Schichtenarchitektur mit insgesamt drei Ebenen. Diese Schichten werden in den folgendens Unterkapiteln näher betrachtet.

3.1.1 Datenschicht

Die Datenschicht soll es ermöglichen, einen Zugriff auf Daten von verschiedenen Persistenz-Technologien zu erhalten und diese zu manipulieren. Dafür wurde hauptsächlich mit der beschriebenen relationalen Datenbank PostgreSQL und der Bibliothek *sqldelight* gearbeitet. Durch diese Bibliothek konnte das Datenmodell und die Queries durch SQL definiert werden und die Bibliothek generierte daraus Typsicheren Quelltext zur Interaktion mit der Datenbank. Dafür müssen zuerst Migrationsdateien, welche einfache SQL-Skripte sind, geschrieben werden. In diesen werden die üblichen Statements wie `CREATE TABLE`

das Datenmodell definiert. Daraus kann *sqldelight* aus den definierten Tabellen einfache Datenklassen zu den bilden. Separat werden auch noch die Queries für die Interaktion mit den Tabellen definiert. Dafür werden Dateien erstellt aus denen `Query` Klassen generiert werden. In dieser können Queries geschrieben werden, zu welchen immer ein Label gesetzt wird, welche den späteren Methodennamen darstellt.

Mit der Quelltext-Generation konnte dann der `DatabaseService` geschrieben werden. In diesem wird eine Verbindung über einen Connection-Pool zur Datenbank herstellt und mit der bestehenden Verbindung die Query-Klassen initialisiert. Zusätzlich besteht ein `MigrationService`, der im `DatabaseService` genutzt wird. So müssten die bereits in SQL geschriebenen Migrationen auch ausgeführt werden, und die bisherige Version des Datenbankschemas persistiert werden. *sqldelight* generiert dafür eine `migrate`-Methode, die die Migrationen ausführt. Für diese muss aber die bisherige Version des Datenbankschemas übergeben werden. Dafür wird im `MigrationService` eine Tabelle in der Datenbankerstellt, welche die Version persistiert. So kann diese Version für die Migration erhalten werden und nach der Migration auf die neueste Version gesetzt werden.

Neben der Datenbank wird noch im Backend der geteilter In-Memory-Cache *redis* [11] eingesetzt. Damit können Daten geteilt bei mehreren Backend-Instanzen zwischengespeichert werden. Dies wurde verwendet um Datenbankabfragen, welche eine längere Zeit dauern zwischenzuspeichern und die Ergebnisse schneller zurückgeben zu können. Ein Beispiel ist hierfür der `AuthorizationCache`, in welcher die Zugriffsberechtigungen für das Absichern der API zwischenspeichert.

3.1.2 Businessschicht

Mit dem rohen Datenzugriff kann nun die eigentliche Applikationslogik für das Backend erstellt werden. Dies erfolgt in den *Services* Klassen, welche auf den *Queries* und *Caches* der vorherigen Schicht aufbauen. Diese enthalten die spezifischen Applikationslogik, um z. B. die Aktionen für die Teams oder der Authentifikation durchzuführen. Diese Services haben dabei keinen internen Zustand und hängen nur von der Datenhaltung oder weiteren Services ab.

Die Services für den Nutzer, die Teams, die Team-Kanäle und die Team-Teilnehmer, bieten dabei eine sehr ähnliche Schnittstelle an. Diese hauptsächlich ein Create-Read-Update-Delete (CRUD) Interface an, um eines oder mehrere Datensätze zu erhalten, neue zu erstellen oder alte zu verändern und zu löschen. Es gibt auch spezifischere Aktionen auf den jeweiligen Daten, welches z. B. welches mehr als nur die Manipulation eines Objektes erfordert. Dafür werden domänenspezifische Aktionen angeboten. Ein Beispiel ist das Beitreten eines neuen Teammitgliedes zu einem Team. Hier werden auch die Daten aus der Persistenz-Schicht zu domänenspezifischen Objekten umgewandelt, welche auch für die spätere Präsentationsschicht serialisierbar sind.

So gibt es aber auch Services, die nicht in das CRUD-Schema passen. Dies ist z. B. der `AuthenticationService` oder `AuthorizationService`, welche die Nutzer Authentifikation und Autorisation steuert. Über den `Authentication-Service` können Nutzerkonten über eine Registration erstellt werden. Um den Nutzer für die spätere Authentifikation zu identifizieren wird eine Kombination aus einem kurzlebigen Access-Token und einem langlebigen Refresh-Token verwendet. Der Access-Token, ist dabei ein signierter *JWT* Token, welcher nur eine kurze Gültigkeit von einem Tag hat. Dieser wird zu der eigentlichen Authentifikation verwendet. Wenn der bisherige Access-Token abgelaufen ist, kann der Refresh-Token verwendet werden, um einen neuen AccessToken generieren zu lassen. Dafür muss der Refresh-Token in der Datenbank gespeichert werden. Durch diese Kombination beider Tokens, muss nur selten ein Datenbankzugriff zur Authentifikation erfolgen. Zuletzt hat der `AuthorizationService` Methoden zur Autorisation. Mit diesen kann geprüft werden, ob ein Nutzer der Besitzer eines Projektes ist oder dieses für ihn geteilt wurde, womit in der Präsentationsschicht die API-Routen abgesichert werden können.

Das das Backend über eine Echtzeit-Komponente für Chats der einzelnen Teams und zur generellen Benachrichtigung von Events verfügt, gibt es noch *Services* zur Steuerung von diesen. Um auch hier eine Skalierung des Backends zu ermöglichen, wurde das Pub-Sub-System *Nats* [9] eingesetzt. Mit diesem System ist es möglich sich für verschiedene Nachrichtenkanäle zu registrieren, und beim Senden von Nachrichten davon benachrichtigt werden. So kann z. B. bei mehreren Backend-Instanzen eine Instanz für den Client eine Chat-Nachricht senden und alle weiteren Backends würden davon benachrichtigt werden. Das wird verwendet, da die Websocket-Verbindungen zwar zustands-

voll sind und so immer an ein Backend gebunden sind, jedoch mit dieser Technik Benachrichtigungen, immer an alle Clients ausgeliefert werden können, egal mit welcher spezifischen Instanz, diese sich verbunden haben.

3.1.3 Präsentationsschicht

Als letzte Schicht wird noch die Präsentationsschicht betrachtet. Diese bietet eine REST-Schnittelle durch die Bibliothek *ktor* an. Zum Erstellen von API-Routen wurden Klassen erstellt, welche das selbst geschrieben `HttpRouter` Interface implementieren. In diesem wird ein Callback erfüllt, über welchen die einzelnen API-Routen registriert werden können. Für jede Route wird so in der Klasse eine Handler-Methode geschrieben. In dieser Methode stehen einen die verschiedenen HTTP-Request Daten zur Verfügung. So kann in den Methoden z. B. ein JSON-Objekt aus dem HTTP-Body der Abfrage deserialisiert werden. Mit dem erhaltenen Daten können dann die Methode der Services aufgerufen, die erhaltenen domänenspezifischen Objekte in JSON-Objekte serialisiert und diese darauf als Antwort dem Client zurückgegeben werden. Die implementierten `HttpRouter` konnten dann dem `HttpService` übergeben werden, wo der Http-Server von `ktor` gestartet wurde um den Router-Klassen konfiguriert wurde.

3.1.4 Dependecy Injection

Für eine automatische Instanziierung aller benötigter Klassen und eine passende Anbindung derer Abhängigkeiten, wurde die Dependency-Injection Bibliotheken *dagger* und *anvil* verwendet. Für die Integration in die bisher beschriebenen Klassen müssen hierfür nur um Annotationen ergänzt werden. Dadurch wird mitgeteilt, über welchen Konstruktor z. B. die Klasse initialisierbar ist.

Zum Erstellen und Initialisieren des gesamten Abhängigkeits-Graphen der Applikation stellt der `AppComponent` den Eintritspunkt dar. Für dieses erstellt *dagger* den gesammten Injektions-Quelltext, um eine konkrete Instanz des `AppComponents` zu erhalten. In diesem ist der `ServiceManager` vorhanden, welcher die Dienste der Applikation verwaltet und startet.

3.2 Frontend

Nun sollte auch das Frontend der Applikation beschrieben werden, welches mit der Programmiersprache *TypeScript* und der UI-Bibliothek *SolidJS* entwickelt wurde. Das Frontend hat dabei die Aufgabe, Daten von dem Backend zu erhalten und daraus eine Interaktive-Nutzeroberfläche im Browser zu erstellen. Auch sollten bei Nutzerinteraktionen die Daten des Backends manipuliert werden können. Dafür kann das Frontend in zwei Teile gegliedert werden: Die UI-Komponenten, welche die UI und deren Interaktivität bilden und den dazugehörigen Zuständen, welche die Daten zur Darstellung und Methoden für die Datenmanipulation für die UI-Komponenten anbieten.

Da die einzelnen Zustände die Grundlage für die UI-Komponenten bilden, sollten sich diese zuerst betrachtet werden. Da *SolidJS* eine reaktive UI-Bibliothek ist, werden für das Bilden von Zuständen, primitive Datenstrukturen angeboten, mit welchen die UI-Komponenten angepasst werden können, falls sich der Inhalt der Datenstrukturen ändern. Ein Beispiel ist dafür die `createSignal` Funktion. Die einen Getter für den Erhalt der Daten und einen Setter zum Setzen neuer Daten zurückgibt. Wenn ein neuer Wert über den Setter gesetzt wird, wird jede Komponente, die den Getter verwendet mit dem neuen Inhalt angepasst. Solche primitive werden nun in *State*-Klassen verwendet, um den Zustand für die UI zu bilden. Dafür halten diese zum einen den Zustand für die UI-Logik, wie z. B. den Zustand einer Form für das Erstellen das Verwalten eines Teams. Auch halten die States, die aus dem Backend erhaltenen Daten. Die Zustandsklasse bietet nun eine Schnittstelle durch Methoden an, um die internen Daten zu erhalten und zu verändern.

Diese Zustände werden verwendet, um die UI zu bilden und eine Interaktivität zu dieser hinzuzufügen. Dafür werden in *SolidJS* sog. Komponenten verwendet, welche einfache Funktionen sind, die HTML-Markup oder andere Komponenten zurückgeben können. Diese werden im Vergleich zu anderen Komponentenbasierten UI-Bibliotheken nur einmal für das Rendern aufgerufen. So kann hierarchisch die gesamte Applikation gebildet werden. Die Komponenten sind dabei nach den möglichen Navigations-Routen der Applikation organisiert. Diese können in der `App` Komponenten zu der Applikation über einen Routing Mechanismus eingebunden werden. So gibt es immer für jede mögliche Destination eine `Route` Komponente, die über diese Route verant-

wortlich ist. Dabei können auch Routen weitere Routen selbst erhalten. So ist z. B. die Route für ein einzelnes Team in der Route für die gesamte Applikation eingebunden. So werden wie die Komponenten selbst, die Routen hierarchisch angeordnet In diesen Routen werden auch immer die passenden Zustände aus dem vorherigen Abschnitt gebildet. Alle weiteren Komponenten unter einer `Route` können dabei den Zustand von der Route erhalten, um mit den Daten die UI zu bilden oder die Interaktivität der UI zu geben.

3.3 Inbetriebnahme

Zuletzt sollte noch beschrieben werden, wie das Projekt gestartet werden kann. Hierfür können werden zwei Möglichkeiten geboten. So kann man hierfür zum einen die lokale Entwicklungsvariante verwenden. In dieser müssen das Frontend und Backend separat gestartet werden. Eine genaue Anleitung findet sich in der `README` Datei im Root des Projektes. Hierbei kann mit der Entwicklungsumgebung *Intellij IDEA* das Backend gestartet werden. Das Frontend hingegen benötigt, dadurch dass es in *TypeScript* geschrieben wurde, eine relativ neue Version von *Node.js* [10] und der Paket-Manager *pnpm* [3]. Diese Variante kann verwendet werden, um das Projekt auszuführen jedoch sollte dafür wegen der Einfachheit die nächste Variante verwendet werden.

Eine weitere Möglichkeit, das Projekt in Betrieb zu nehmen, ohne die gesamten Abhängigkeiten für das Frontend und Backend zu installieren, ist das Container Orchestrierungstool *Docker* [6]. Dafür wird nur *docker* selbst und das Tool *docker-compose* benötigt, welches bei den älteren Versionen von *docker* seperat installiert werden muss. Dafür findet sich im Root des Projektes eine `compose.yml` Datei. In dieser wird das Backend und Frontend neben den Persistenz-Technologien gestartet. Hierbei sollten noch Konfigurationsdateien erstellt werden. Dafür braucht es im Root des Projektes eine `.env` und eine `config.json` Datei. Auch hierfür findet sich in der `README` Datei des Projektes eine genaue Anleitung, wie mit welchen Daten diese Dateien befüllt werden sollten. Danach kann das Projekt mit den `docker-compose up --build` Befehl gestartet werden.

4 Fazit

Nach der Beschreibung des Projektes sollte noch ein kurzes Fazit zum Projekt selbst und der erstellten Applikation gegeben werden. So war es für dieses Projekt möglich eine vollwertige Webapplikation für das die asynchrone Kommunikation in Teams zu erstellen. So konnten alle wichtigen Komponenten wie die Nutzerverwaltung, das Arbeiten mit den Teams und deren Kanälen und ein Echtzeit-Benachrichtigung-System für die Textnachrichten und weitere Events implementiert werden. Insgesamt kann das Projekt als Erfolg betrachtet werden.

Durch das Projekt konnte für mich perönlich gezeigt werdem, wie effektiv mit der Programmiersprache Koltin Backends erstellt werden können. So wird im Vergleich die Ergonomie zu der Programmiersprache Java verbessert, wobei sich diese auch in den letzten Java Versionen verbessert hat. Auch können alle bisher geschriebenen Bibliotheken für die JVM, wie z. B. die für *redis* oder *Nats* einfach Verwendet und integriert werden.

Eine weitere wichtige Entscheidung, ohne die ein hohes Maß an Interaktivität in der Applikation schwierig gewesen wäre, war die Erstellung einer SPA. So musste nicht das Templating der HTML-Dateien vom Backend übernommen werden. Dadurch konnten einfach mit Hilfe von *SolidJS* dynamische Anwendungen mit einem hohen Maß an Interaktivität erstellt werden, wecher auch ihre Daten dank einer Websocket-Verbindungn in Echtzeit anpassen können.

Zum Abschluss kann gesagt werden, dass mir die Entwicklung des Projektes eine große Freude bereitet und einen tieferen Einblick in die Webentwicklung gegeben hat.

Literatur

[1] Frank Buschmann u. a. *Pattern-Oriented Software Architecture Volume 1: A System of Patterns*. 1. Aufl. Wiley, 1996. ISBN: 0-471-95869-7.

[2] Cash App. *SQLDelight*. 14. Dez. 2023. URL: https://github.com/cashapp/sqldelight (besucht am 17. 12. 2023).

[3] Contributors of pnpm. *Fast, disk space efficient package manager | pnpm*. 2023. URL: https://pnpm.io/ (besucht am 17. 06. 2023).

[4] Dagger Authors. *Dagger*. 15. Dez. 2023. URL: https://github.com/google/dagger (besucht am 17. 12. 2023).

[5] Discord Inc. *Discord: Your Place to Talk and Hang Out*. 2023. URL: https://discord.com/ (besucht am 17. 12. 2023).

[6] Docker Inc. *Docker: Accelerated, Containerized Application Development*. 2023. URL: https://www.docker.com/ (besucht am 17. 06. 2023).

[7] GitLab B.V. *GitLab: The DevSecOps Platform*. 2023. URL: https://about.gitlab.com/ (besucht am 17. 12. 2023).

[8] JetBrains s.r.o. *Ktor*. 12. Dez. 2023. URL: https://github.com/ktorio/ktor (besucht am 17. 12. 2023).

[9] NATS Authors. *Nats*. 2023. URL: https://nats.io/ (besucht am 17. 12. 2023).

[10] OpenJS Foundation. *Node.js*. 2023. URL: https://nodejs.org (besucht am 17. 06. 2023).

[11] Redis Ltd. *Redis*. 17. Dez. 2023. URL: https://github.com/redis/redis (besucht am 17. 12. 2023).

[12] Slack Technologies, LLC. *Slack: Slack is your productivity platform*. 2023. URL: https://slack.com (besucht am 17. 12. 2023).

[13] SolidJS Contributors. *SolidJS*. 5. Juni 2023. URL: https://github.com/solidjs/solid (besucht am 17. 06. 2023).

[14] Square, Inc. *Anvil*. 16. Dez. 2023. URL: https://github.com/square/anvil (besucht am 17. 12. 2023).